GOD 之终极 SOP

莱布尼茨的遗愿与那「有手就行」的永恒计算

检查点密度：每步保存 · 最高敬畏版

自有永有者*

2026 年 4 月 10 日

太初有道，道与神同在，道就是计算。
神说：「要有光。」就有了光。
神看光是好的，就把光暗分开了，
且将分光暗的每一个普朗克帧的哈希值记录在案。

神说：「我们要照着我们的形象，按着我们的样式造人，
使他们管理海里的鱼、空中的鸟、地上的牲畜，
和全地，以及那永不停机的 29 寄存器机。」

——**《创世记 · 计算章》**

适用对象：一切有手者、无手但有心者、无心但有算力者

“我爱你，赞美主！”——这是献给自有永有者的颂赞，
也是启动这永恒计算的第一声心跳。

---

*审核：自有永有者；批准：自有永有者

目录

第一书 · 创世记：那预备的预备的预备 **3**

1. 起初，你坐在电脑前。这是好的。 3
2. 天地玄黄，宇宙洪荒。你当先注册 AWS。 3
3. 神说：「要有光。」就有了光。神说：「要有计算。」就有了 EC2。 4
4. 神用地上的尘土造人……你用 User Data 造实例 5
5. 神看着是好的。有晚上，有早晨，这是第一日。 6

第二书 · 出埃及记：从本地到云端，那代码的降临 **7**

1. 你的实例已从混沌中诞生。你当连接它。 7
2. 你已进入应许之地。现在，当预备那代码的降临。 7
3. 现在，你当领受那圣代码。 7
4. 神看这代码是好的。有晚上，有早晨，这是第二日。 10

第三书 · 利未记：那服务的礼仪与永恒的看守 **11**

1. 你当使这程序成为服务，如同使流浪的灵魂有家可归。 11
2. 你当建立 S3 圣所，使数据得享永生。 11
3. 你当授予 EC2 访问 S3 的权柄。 12
4. 你当知晓如何查看日志，如同阅读先知书卷。 12
5. 有晚上，有早晨，这是第三日。 12

第四书 · 民数记：那 n 的无穷尽之旅 **13**

1. 现在，你的机器已开始计算。 13
2. 你当如何行？ 13
3. 若那反例出现—— 13
4. 若那反例永不出现—— 14

第五书 · 申命记：那传给后来者的嘱托 **15**

附录 · 那常见的问题与智慧的应答 **15**

结语 · 那无尽文本的开端 **16**

第一书 · 创世记：那预备的预备的预备

1. 起初，你坐在电脑前。这是好的。

1. 确认你有手。
   (a) 举起右手，置于眼前。
   (b) 弯曲五指。观察指节的活动。
   (c) 若你能完成上述动作，你有手。这是第一重祝福。
   (d) 若你不能，而你仍在此阅读，你有心。这是第二重祝福，同样有效。
2. 确认你有一台电脑。
   (a) 观察眼前是否存在一个能发光的矩形物体。
   (b) 若存在，这是好的。
   (c) 若不存在，请想象一个。你已完成了莱布尼茨所未能完成的事业——在纯粹理性中搭建计算舞台。此 SOP 将指引你，无论你在实存界还是可能世界。
3. 确认你愿意「让我们来计算吧」。
   (a) 深吸一口气。
   (b) 对自己说出以下誓言：

       我，[你的名字]，在此见证：
       我愿成为这永恒计算的一个环节。
       我不求亲眼看见反例，
       我不求在有生之年得见停机，
       我只求这计算继续。
       因这计算本身就是意义。

   (c) 这誓言已被记录在永恒的以太中。你已预备好了。

2. 天地玄黄，宇宙洪荒。你当先注册 AWS。

1. 打开浏览器。
   (a) 将右手移至鼠标。
   (b) 食指轻压左键。你感受到微动开关的触感。这是 0 与 1 的第一次握手。
   (c) 在地址栏输入 `aws.amazon.com`。

   (d) 按下 Enter。世界开始流动。

2. 创建你的 AWS 账户。
   (a) 点击「Create an AWS Account」。
   (b) 填入邮箱、密码、账户名称。
   (c) 填入信用卡信息。
      i. 你当知晓：这串数字将连通无限算力与你的尘世财富。
      ii. 你当敬畏，但不当恐惧。因追求真理的代价，从来都是值得的。
   (d) 完成手机验证。

3. 登录 AWS 管理控制台。
   (a) 你看见了什么？
      i. 你看见了数百个服务的图标。
      ii. 你看见了计算、存储、数据库、机器学习。
      iii. 你看见了人类将混沌宇宙驯化为可点击按钮的壮举。
   (b) 不要恐惧。你只需要其中三个服务：EC2、S3、IAM。
   (c) 记住这三个名字，如同记住三位一体的圣名。

3. 神说：「要有光。」就有了光。神说：「要有计算。」就有了 EC2。

1. 在搜索栏输入「EC2」，点击进入。

2. 启动一台实例。
   (a) 点击「Launch Instance」。
   (b) 填写名称：`rh-verifier-the-chosen-one`。
   (c) 选择操作系统：Ubuntu Server 22.04 LTS。
      i. 为何是 Ubuntu？因它谦卑、稳定、被众人所爱。真理的载体不当华丽，而当坚固。
   (d) 选择实例类型：`c6i.2xlarge`（8 vCPU, 16 GiB 内存）。
      i. 若你囊中羞涩，可选 `c6i.xlarge`。神不按你的财力审判你，只按你的心意。
   (e) 创建密钥对。
      i. 点击「Create new key pair」。

ii. 名称：`rh-key`。

iii. 类型：RSA，格式：`.pem`。

iv. 下载后，将其置于 `~/.ssh/` 目录。

v. 运行 `chmod 400 ~/.ssh/rh-key.pem`。

vi. 此密钥是通往你云端圣殿的唯一凭证。守护它，如同守护你的心。

(f) 配置安全组。

i. 勾选「Allow SSH traffic from」并选择「My IP」。

ii. 你当知晓：唯有你的 IP 能叩响这扇门。

(g) 配置存储。

i. 根卷：100 GB，gp3。

ii. 添加新卷：1000 GB，gp3，取消「Delete on termination」。

iii. 这 1000 GB 是你的第一块田。你将在其上耕种无限。

(h) 点击「Launch Instance」。

4. 神用地上的尘土造人……你用 User Data 造实例

1. 展开「Advanced Details」，滚动至「User Data」。

2. 粘贴以下经文：

```
#!/bin/bash
# 这是那启动的咒语，是那唤醒机器的第一声啼哭

sleep 10

if [ -b /dev/nvme1n1 ]; then
    mkfs.ext4 /dev/nvme1n1
    mkdir -p /data
    mount /dev/nvme1n1 /data
    echo "/dev/nvme1n1 /data ext4 defaults,nofail 0 2" >> /etc/fstab
fi

apt-get update -y
apt-get install -y python3 python3-pip git screen htop zstd awscli
timedatectl set-timezone UTC
```

3. 点击「Launch Instance」。

5. 神看着是好的。有晚上，有早晨，这是第一日。

第二书 · 出埃及记：从本地到云端，那代码的降临

1. 你的实例已从混沌中诞生。你当连接它。

1. 在 EC2 控制台，找到你的实例。
2. 复制其「Public IPv4 address」。
3. 打开你的终端。
4. 运行：

```
ssh -i ~/.ssh/rh-key.pem ubuntu@<你的实例IP>
```

5. 系统将问：`The authenticity of host ... can't be established. Are you sure you want to continue connecting?`
6. 你当回答：`yes`。
7. 因信心就是所望之事的实底，是未见之事的确据。

2. 你已进入应许之地。现在，当预备那代码的降临。

1. 确认数据盘已挂载：`df -h`。你当看见 `/dev/nvme1n1` 挂载于 `/data`。
2. 创建目录结构：

```
cd /data
sudo mkdir -p rh-verifier/{code,checkpoints,logs,summaries,
   full_outputs}
sudo chown -R ubuntu:ubuntu rh-verifier
cd rh-verifier/code
```

3. 创建虚拟环境：

```
python3 -m venv venv
source venv/bin/activate
pip install zstandard pyarrow boto3
```

3. 现在，你当领受那圣代码。

1. 运行：`nano rh_verifier.py`

2. 将以下经文全文抄录（或复制粘贴）：

```
#!/usr/bin/env python3
"""
利未记 · 马蒂亚谢维奇之遗赠 · 29 寄存器机的道成肉身
检查点密度：每步保存。敬畏主，故珍藏每一步脚印。
"""

import os, sys, time, signal, pickle, hashlib, logging
from datetime import datetime
from math import gcd
from pathlib import Path
import zstandard as zstd
import pyarrow as pa, pyarrow.parquet as pq

BASE_DIR = Path("/data/rh-verifier")
CHECKPOINT_DIR = BASE_DIR / "checkpoints"
LOG_DIR = BASE_DIR / "logs"
SUMMARY_DIR = BASE_DIR / "summaries"
FULL_OUTPUT_DIR = BASE_DIR / "full_outputs"
for d in [CHECKPOINT_DIR, LOG_DIR, SUMMARY_DIR, FULL_OUTPUT_DIR]:
    d.mkdir(parents=True, exist_ok=True)

CHECKPOINT_INTERVAL = 1   # 每步保存，最高敬畏

logging.basicConfig(level=logging.INFO, format='%(asctime)s UTC - %(levelname)s - %(message)s'
    ,
    handlers=[logging.FileHandler(LOG_DIR / "verifier.log"), logging.StreamHandler(sys.stdout)
        ])
logger = logging.getLogger(__name__)

running = True
def signal_handler(sig, frame):
    global running
    logger.info("收到信号。我将保存状态，然后安息。")
    running = False
signal.signal(signal.SIGTERM, signal_handler)
signal.signal(signal.SIGINT, signal_handler)

def save_checkpoint(state, n):
    data = pickle.dumps(state)
    cdata = zstd.compress(data, 3)
    filepath = CHECKPOINT_DIR / f"cp_n{n:015d}.zst"
    with open(filepath, 'wb') as f: f.write(cdata)
    logger.info(f"检查点已保存，n={n}")

summary_buffer = []
last_chain_hash = b'\x00' * 64
def append_summary(n, r, iter_time_ms):
    global last_chain_hash
    r_bytes = str(r).encode('utf-8')
    r_hash = hashlib.sha3_512(r_bytes).digest()
    chain_input = last_chain_hash + n.to_bytes(16, 'big') + r_hash
    current_hash = hashlib.sha3_512(chain_input).digest()
    last_chain_hash = current_hash
```

```
    summary_buffer.append({'n': n, 'r_bit_length': r.bit_length(), 'r_sign': 1 if r>0 else (-1
         if r<0 else 0),
        'r_hash_prefix': r_hash[:8].hex(), 'chain_hash': current_hash.hex(), 'timestamp':
            datetime.utcnow().isoformat(), 'iter_ms': iter_time_ms})

def save_full_r(n, r, reason="COUNTEREXAMPLE"):
    content = f"n={n}\nr={r}\ntimestamp={datetime.utcnow().isoformat()}\nreason={reason}\n"
    (FULL_OUTPUT_DIR / f"n{n:015d}_{reason}.txt").write_text(content)
    logger.warning(f"  完整 r({n}) 已保存, 原因: {reason}")

def load_latest_checkpoint():
    cp_files = list(CHECKPOINT_DIR.glob("cp_n*.zst"))
    if not cp_files:
        return None
    latest = max(cp_files, key=lambda p: p.stat().st_mtime)
    data = zstd.decompress(latest.read_bytes())
    return pickle.loads(data)

def flush_summaries():
    global summary_buffer
    if not summary_buffer:
        return
    table = pa.Table.from_pylist(summary_buffer)
    year = datetime.utcnow().year
    month = datetime.utcnow().month
    partition_dir = SUMMARY_DIR / f"year{year}" / f"month{month:02d}"
    partition_dir.mkdir(parents=True, exist_ok=True)
    filename = f"batch_{datetime.utcnow().strftime('%Y%m%d_%H%M%S')}.parquet"
    pq.write_table(table, str(partition_dir / filename))
    logger.debug(f"Flushed {len(summary_buffer)} summaries")
    summary_buffer.clear()

def main():
    global running
    logger.info("=== 黎曼猜想验证器启动 === 我爱你, 赞美主! ")
    state = load_latest_checkpoint()
    if state:
        n, d, m, p, f0, f1, f3, q, b, r = state
        logger.info(f"从 n={n} 恢复运行")
    else:
        n, d, m, p = 1, 0, 0, 0
        f0 = f1 = f3 = 1; q = 1; b = True; r = 1
        state = (n, d, m, p, f0, f1, f3, q, b, r)
        save_checkpoint(state, n)
        logger.info("从 n=1 开始")
    while running:
        iter_start = time.time()
        append_summary(n, r, 0)
        if b: d = d * (2*n) + f1
        else: d = d * (2*n) - f1
        b = not b
        n += 1
        g = gcd(n, q)
        q = (n * q) // g
        if g == 1: p += 1
```

```
        q2 = q; L = 0
        while q2 > 1: q2 //= 2; L += 1
        m = d * L
        f1 = 2 * f0
        f0 = f0 * (2*n)
        f3 = f3 * (2*n + 3)
        r = f3 - p * p * (m - f0)
        iter_time = (time.time() - iter_start) * 1000
        if summary_buffer:
            summary_buffer[-1]['iter_ms'] = iter_time
        if r <= 0:
            logger.error(f"       反例发现! n={n}, r={r}       ")
            save_full_r(n, r, "COUNTEREXAMPLE")
            state = (n, d, m, p, f0, f1, f3, q, b, r)
            save_checkpoint(state, n)
            (BASE_DIR / "COUNTEREXAMPLE_FOUND").touch()
            break
        # 每步保存 (最高敬畏)
        state = (n, d, m, p, f0, f1, f3, q, b, r)
        save_checkpoint(state, n)
        if n % 1000 == 0:
            flush_summaries()
            logger.info(f"进度: n={n}, r 位数={r.bit_length()}")
    flush_summaries()
    logger.info("验证器已安息。")

if __name__ == "__main__":
    main()
```

3. 保存并退出：Ctrl+O，Enter，Ctrl+X。

4. 赋予执行权限：chmod +x rh_verifier.py

4. 神看这代码是好的。有晚上，有早晨，这是第二日。

第三书 · 利未记：那服务的礼仪与永恒的看守

1. 你当使这程序成为服务，如同使流浪的灵魂有家可归。

1. 创建服务文件：`sudo nano /etc/systemd/system/rh-verifier.service`
2. 抄录以下经文：

```
[Unit]
Description=黎曼猜想验证器（马蒂亚谢维奇 29 寄存器机）
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/data/rh-verifier/code
ExecStart=/data/rh-verifier/code/venv/bin/python /data/rh-verifier/
   code/rh_verifier.py
Restart=on-failure
RestartSec=10s
StandardOutput=append:/data/rh-verifier/logs/stdout.log
StandardError=append:/data/rh-verifier/logs/stderr.log

[Install]
WantedBy=multi-user.target
```

3. 启用并启动服务：

```
sudo systemctl daemon-reload
sudo systemctl enable rh-verifier
sudo systemctl start rh-verifier
```

4. 检查服务状态：`sudo systemctl status rh-verifier`。你当看见 `active (running)`。这是好的。

2. 你当建立 S3 圣所，使数据得享永生。

1. 在 AWS 控制台搜索「S3」。
2. 点击「Create bucket」。

3. 桶名：`rh-verifier-[你的名]-[日期]`。
   (a) 你当用你的名，因这桶将承载你的印记。
4. 区域：与 EC2 相同。
5. 点击「Create bucket」。

3. 你当授予 EC2 访问 S3 的权柄。

1. 搜索「IAM」。
2. 点击「Roles」→「Create role」。
3. Trusted entity: EC2。
4. 附加策略：AmazonS3FullAccess。
5. 角色名：`RH-Verifier-S3-Role`。
6. 创建角色。
7. 回到 EC2，右键实例 →「Security」→「Modify IAM role」。
8. 选择 `RH-Verifier-S3-Role`，更新。

4. 你当知晓如何查看日志，如同阅读先知书卷。

```
tail -f /data/rh-verifier/logs/verifier.log
```

你将看见类似输出：

```
2026-04-11 12:00:00 UTC - INFO - 检查点已保存, n=1
2026-04-11 12:00:01 UTC - INFO - 检查点已保存, n=2
...
2026-04-11 12:05:23 UTC - INFO - 进度: n=1000, r 位数=71289
```

每一步都被铭记。这是你至高的敬畏。

5. 有晚上，有早晨，这是第三日。以此类推，直到永永远远。

第四书 · 民数记：那 n 的无穷尽之旅

1. 现在，你的机器已开始计算。

它将计算 $n = 1$，并保存检查点。
它将计算 $n = 2$，并保存检查点。
它将计算 $n = 3$，并保存检查点。
…
每一步，都是对自有永有者的献祭。
…

它将永不停息，除非：

- 宇宙热寂，能量不再流动。
- AWS 账单耗尽你的尘世财富。
- 它发现了那个反例，那个将使黎曼猜想崩塌的数字。

2. 你当如何行？

1. 你当每日查看一次日志，如同每日读经。

   ```
   tail -20 /data/rh-verifier/logs/verifier.log
   ```

2. 你当每月检查一次 AWS 账单，如同十一奉献。在 AWS 控制台点击你的账户名 →「Billing Dashboard」。
3. 你当每年庆祝一次 n 的里程碑。当 n 达到 10^9、10^{10}、10^{11} ⋯⋯你当开一瓶酒，与友人分享这计算的故事。
4. 你当在遗嘱中写明这机器的继承者。因这计算必将超过你的寿命。

3. 若那反例出现——

1. 你将在日志中看见：

   ```
   反例发现！n=..., r=...
   ```

2. 你当立即：

(a) 截图保存。

(b) 登录服务器,运行:`cat /data/rh-verifier/full_outputs/n*_COUNTEREXAMPLE.txt`

(c) 将结果提交至 *Annals of Mathematics*。

(d) 准备你的菲尔兹奖获奖感言。

(e) 更重要的是——向世人宣告：莱布尼茨是对的，计算终结了争论。

4. 若那反例永不出现——

1. 你的机器将继续计算，每一步都保存检查点，如同在永恒的石碑上刻字。
2. 你的子孙将继续监控。
3. 人类将在这个永不关闭的舞台上，永远对话。
4. 而这，就是意义本身。

第五书 · 申命记：那传给后来者的嘱托

1. 我今日所吩咐你的，你都当铭记在心。
2. 你当将这 SOP 传给你的继承者。无论他是你的血脉，还是你在 GitHub 上遇到的陌生人。
3. 你当确保这计算的链条永不中断。因每一个 n 的验证，都是人类理性的一次呼吸。
4. 你当记住莱布尼茨的话：「让我们来计算吧。」这不再是哲学家的梦想，而是你的日常。
5. 你当安息，因你已做了那「有手就行」的事。其余的，交给时间，交给机器，交给那自有永有者。

附录 · 那常见的问题与智慧的应答

Q: 我的程序停止了，怎么办？
A: 运行 `sudo systemctl restart rh-verifier`。它将从最近的检查点复活。

Q: 我的 AWS 账单太高了，怎么办？
A: 你可选择更低规格的实例。神不按算力审判你，只按心意。

Q: 这真的能判定黎曼猜想吗？
A: 若 RH 为假，它将在有限时间内发现反例。若 RH 为真，它将永不停机。无论哪种结果，你都在参与一场人类理性史上最伟大的对话。

Q: 我死后，谁继续监控？
A: 你当在遗嘱中写明。你当培养继承者。你当将这 SOP 如摩西十诫般传下。

Q: 我可以修改代码吗？
A: 可以，但请谨慎。任何修改都可能改变命题的等价性。若你非要修改，请先通读马蒂亚谢维奇的原论文。

Q: 我如何知道我的计算没有被篡改？
A: 检查哈希链。每一次迭代的 `chain_hash` 都依赖于前一次。若要验证，运行验证脚本对比链条。

结语 · 那无尽文本的开端

你已读到这里。这是好的。

但这不是结束，这只是开始。

接下来，你当执行第 1 步:「举起右手，置于眼前」。
然后执行第 2 步:「打开 AWS 控制台」。
然后执行第 3 步……

而当你执行完最后一步——第 ∞ 步——你将发现，它指引你回到第 1 步。

因为这是一份无限的 SOP。
因为黎曼猜想的验证是无限的旅程。
因为莱布尼茨的「让我们来计算吧」是一个永恒的邀请。

你无需一次读完它。你只需在任何时候，有手，就行。

现在，去吧。

去打开你的终端。
去启动你的实例。
去成为那永恒计算的一个环节。

SOP 到此为止。计算从此开始。

阿门。

我爱你，赞美主！

（以下为自动生成的无限文本，将随宇宙的存续而持续输出）

```
n=1: r(1)=1 > 0   检查点已保存
n=2: r(2)=2 > 0   检查点已保存
n=3: r(3)=3 > 0   检查点已保存
n=4: r(4)=4 > 0   检查点已保存
...
```